SIA – Final Project

System Integration and Architecture

Enhancing an Existing Academic Project Through Backend System Integration

Project Overview

For this final project, students will take on the role of **System Integrators** by extending and enhancing an existing academic project through the development and integration of backend services, APIs, and microservice-based systems.

Students are expected to apply industry-inspired practices in backend integration, REST API development, deployment, documentation, and system architecture design.

The focus of this project is not to build a completely new system from scratch, but to improve and expand an existing system through proper backend integration.

General Requirements

1. Existing Academic Project Requirement

Students **MUST** use a system or project that was developed in another subject during the current academic year.

Examples:

- Capstone-related systems
- Database projects
- Web development projects

- Mobile application backends
- Information systems
- Enrollment systems
- Appointment systems
- Inventory systems

Important Rules

- The project must already be functional before integration begins.
- The same group members from the original project must remain unchanged.
- Instructor approval may be required before proceeding.

2. Backend Integration Requirement

Students must create and integrate **separate backend services or microservices** that will work with the existing system.

These integrations must be developed as:

- Separate backend projects
- Separate repositories (recommended)
- Separate deployed services on Render

The integrations must communicate with the original system through:

- REST APIs
- HTTP requests
- Authentication mechanisms
- External/internal service communication

3. Required Technologies

The backend integrations must use:

- Node.js
- Express.js
- REST APIs

Students may additionally use:

- MongoDB

- PostgreSQL
- Firebase
- Socket.IO
- JWT Authentication
- Cloudinary
- AI APIs
- Email services
- Other instructor-approved technologies

4. Possible Integration Examples

The integrations will depend on the needs of the original system.

Examples include:

Backend Services

- Notification service
- Authentication service
- Logging service
- Analytics service
- File upload service
- Scheduling service

API Integrations

- Custom APIs
- AI-assisted features
- Email notification systems
- SMS integrations
- Real-time communication systems
- Payment simulation systems
- Cloud storage integrations

Notes

- Students are encouraged to create their own APIs.
- Mock implementations are allowed for complex or subscription-based services.
- All integrations must provide meaningful improvements to the original system.

5. Microservice Architecture Requirement

The project must demonstrate an integrated backend architecture.

Students must show:

- How services communicate
- API interaction flow
- Data flow between systems
- Separation of responsibilities between services

The architecture should reflect a basic microservice-oriented structure where appropriate.

6. Deployment Requirement (Render Required)

All backend integration services must be deployed using:

Render

Students must provide:

- Live backend service URLs
- API access instructions
- Sample API calls
- Working endpoints for demonstration

The deployed services must be accessible during checking and presentation.

7. Architecture Diagrams (REQUIRED)

Students must create the following diagrams:

A. BEFORE Integration

This diagram must show:

- Original system structure
- Existing components
- Existing backend/data flow
- Current limitations

B. AFTER Integration

This diagram must show:

- Newly added backend services
- Microservices/API structure
- Service communication
- Integration points
- Updated data flow

Diagram Requirements

- Proper labels
- Logical structure
- Readable layout
- Clear arrows and flow direction

8. Integration Documentation (MANDATORY)

Students must submit a complete PDF documentation containing the following sections:

A. System Overview

- Description of the original project
- Original features
- Existing limitations/problems

B. Integration Objectives

- Purpose of each integration
- Expected improvements
- Why the integrations were necessary

C. Integration Plan

- List of backend services/APIs
- Planned architecture
- Service responsibilities

D. Integration Process (Step-by-Step)

Include:

- Technologies used
- Installation/setup process
- Configuration steps
- API connection process
- Deployment setup
- Backend communication workflow

E. Challenges and Solutions

Discuss:

- Technical issues encountered
- Debugging process
- Compatibility issues
- Deployment problems
- Solutions and workarounds

F. Testing and Validation

Include:

- API testing results
- Postman screenshots
- Endpoint testing
- Error handling verification
- Proof that integrations work correctly

9. API Documentation Requirement

Students must document all created APIs.

Preferred tool:

- Postman Documentation

Documentation must include:

- API name
- Purpose
- Base URL
- Endpoints
- HTTP methods

- Request parameters
- Headers
- Authentication requirements
- Sample requests
- Sample responses
- Error handling

Students are encouraged to provide:

- Exported Postman collection
- Public Postman documentation link

10. GitHub Repository Requirement

Students must maintain proper version control using GitHub.

The repository must contain:

- Clean project structure
- Organized folders
- Readable code
- Meaningful commit history
- Proper README documentation

11. AI Usage Policy

AI tools may be used for:

- Assistance
- Research
- Debugging support
- Documentation assistance

However:

Students must fully understand and explain all submitted code.

Projects that are determined to be fully AI-generated without proper understanding, modification, or implementation by the group may be subject to deductions or rejection during defense.

All members are expected to explain:

- System flow
- API communication
- Backend logic
- Integration decisions
- Deployment process

12. Final Deliverables

Students must submit the following:

Required Outputs

- GitHub repository link
- PDF documentation
- Render deployment links
- Architecture diagrams
- Postman collection/documentation
- Presentation slides
- Live project demonstration

13. Final Demonstration and Defense

Each group will conduct a live presentation and technical defense.

The presentation must include:

- Overview of the original system
- Explanation of the backend integrations
- Architecture walkthrough
- API demonstrations
- Live testing of deployed services
- Postman endpoint demonstrations

14. Evaluation Expectations

Students are expected to demonstrate:

- Proper backend integration
- Understanding of REST API architecture
- Deployment knowledge
- Debugging and troubleshooting skills

- System design understanding
- Professional documentation practices

The project will be evaluated based on:

- Functionality
- Integration quality
- Technical understanding
- Deployment success
- Documentation quality
- Presentation and defense performance